

claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_887fb14e-e6d\agent-a91434421ecaddcb8.jsonl`  
- SHA-256: `114bad65bc30e776d7f3d11a10958826b46d7f0fb6f712a203b8d1d09e4f5b33`  
- Source modified: `2026-06-20T10:17:00+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_887fb14e-e6d`  
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`
```

Transcript

1. user (2026-06-20T10:12:41.341Z)

Consolidate research into ONE tracked design doc for the rally-annotator integration + "train your own model" USP pages in KhelSutra Studio.

RESEARCH (ground truth):

```
{  
  "annotator": {  
    "what_it_is": "[verified] rally-annotator is a single-file VLC Lua EXTENSION (not a standalone app) that lets a human hand-label rally start/end + a point-ending reason while watching a match in VLC, appending one CSV row per rally. Targets net-separated racquet sports (badminton, tennis, table tennis, pickleball, padel) which share a forced/unforced-error taxonomy, so one tool covers all five. README.md:5-10;  
vlc/rally_annotator.lua:1-12.\n\n[verified] Core file vlc/rally_annotator.lua (~860 lines; VERSION constant = 1.6.4 at line 88). Declares NO VLC listener capabilities (descriptor capabilities = {}) at line 111); it is a button-driven dialog whose callbacks SNAPSHOT the current playback time on click. Deliberate: the VLC auto-pause/playing-listener callback is flaky on macOS (VLC #22778) and broken in VLC 4.0 (#27558), so a persistent button dialog is portable and frame-precise (lines 63-70). Playback time read via vlc.var.get(input, 'time') which is MICROSECONDS in VLC 3.x, /1e6 to seconds (now_seconds, lines 201-207).\n\n[verified] BYO-model pipeline positioning: the output CSV is intentionally ingest-compatible with the badminton-highlight-indexer loaders, which read by header NAME (rally_number, start_time, end_time) and drop rows where end <= start. docs/CSV_FORMAT.md:27-44 documents exact ingest commands: 'python -m backend.eval.calibrate_wasb --labels match.rallies.csv', 'python -m backend.tools.rally_slicer --labels match.rallies.csv', and a sports-data-collector import 'python -m src.cli.curate import-local ... --annotations-path match.rallies.csv'. v1.1 (CHANGELOG.md:133-137) says the tool ORIGINATED inside badminton-highlight-indexer before being spun out as this MIT standalone. So it is literally the 'annotate your footage -> golden labels -> train/eval your own rally model' entry point.\n\n[verified] Stated value prop (README.md:22-24): labeling rally boundaries is the slow, expensive prerequisite for any rally-detection model; this lets a rater do it inside the player they already use, pausing to the exact frame before each mark, turning 'watch a match' into 'produce golden labels'. \n\n[design - my USP framing, not asserted by repo] For the Studio UI: this is the concrete real artifact behind the 'bring-your-own-model moat' - users self-produce clean, MIT-licensed ground-truth that never depends on a paid LLM, feeding the owned-model track. HONEST status to preserve: rally-annotator is REAL and shipping (v1.6.4); the downstream train-your-own-model loop in badminton-highlight-indexer is largely ROADMAP/in-progress and owner-gated, so Studio pages must NOT imply one-click training exists today.",  
    "install": "[verified] Installed as a VLC Lua extension by copying ONE file into VLC's per-user Lua 'extensions' folder (create it if missing). README.md:26-35;  
vlc/rally_annotator.lua:75-80.\n\nStep 1 - copy vlc/rally_annotator.lua into the extensions folder:\n  - Windows: %APPDATA%\vlc\lua\extensions\\\n  - macOS: ~/Library/Application Support/org.videolan.vlc/lua/extensions/\n  - Linux: ~/.local/share/vlc/lua/extensions/\nStep 2 - in VLC: Tools -> Plugins and extensions -> Reload extensions (or just restart VLC).\nStep 3 - enable it from the View menu -> Rally Annotator. A small dialog opens and stays open while you watch.\n\n[verified] Requires VLC 3.0.x. VLC 4.0 changed the Lua input/listener API, so the tool
```

targets 3.x for now (README.md:35).\n\n[verified] Confirm which build is loaded: VLC's Active Extensions tab shows the title verbatim WITH the version baked in, e.g. 'Rally Annotator v1.6.4' (version is concatenated into descriptor().title; vlc/rally_annotator.lua:90-97, 776-777). VLC's own About box body ('Lua script') is hardcoded by VLC and CANNOT be set by an extension - use the in-dialog Help button instead (README.md:68-72; vlc/rally_annotator.lua:54-61).\n\n[verified] No package manager / no build step - plain Lua, single file, zero runtime dependencies. End users do NOT need Lua installed. For DEVELOPERS only, the headless test suite needs Lua 5.1 (the interpreter VLC 3.x embeds); test/README.md:21-42 gives per-OS install incl. a no-admin portable Windows LuaBinaries 5.1.5 route and 'choco install lua51'.",

"annotate_workflow": "[verified] Workflow (README.md:37-66; vlc/rally_annotator.lua dialog + callbacks). Playback is built INTO the annotation window so you never switch to the main VLC window: a row of Back 5s / Play / Pause / Fwd 5s (lines 784-787). Play/Pause is ONE toggle that branches on vlc.playlist.status() so it never flips the wrong way (play_pause, lines 739-753); seek is relative +/-5s clamped at 0 (seek_by, lines 593-603).\n\n1. Pick the Sport dropdown (top-left). It stays set across rallies (default badminton; w_sport, lines 780-781).\n2. When a rally begins, click 'Mark START' - snapshots the current playback time into the editable 'Start (s)' field (pause/scrub first for frame accuracy; you can also type the value). mark_start, lines 608-620.\n3. When the rally ends, click 'Mark END' - fills the editable 'End (s)' field. This only ARMS the rally; nothing is written yet. mark_end, lines 622-628.\n4. Choose 'Ending reason' (dropdown sitting between Mark END and Save Rally; default 'unknown'), optionally type 'Number of shots' (blank to skip), then click 'Save Rally' to write ONE CSV row. save_rally, lines 630-671.\n5. After every save the reason RESETS to 'unknown' and the shots field clears (non-sticky) so a forgotten pick records 'unknown', never the previous rally's value (reset_form, lines 527-536; README.md:48-50). Tolerates reversed marks (swaps if end<start) and refuses a zero/negative-length rally (lines 636-639).\n\n[verified] Editing / safety: 'Recent rallies' list shows EVERY rally oldest-first, scrollable (refresh_list, lines 447-456). Select a row then 'Edit selected' (loads it back; Mark buttons relabel to 'Re-mark START/END (#N)', Save becomes 'Save changes (#N)') or 'Delete selected'. 'Undo last' removes the most recent row (button shows which, e.g. 'Undo last (#7)'), or clears an in-progress mark, or cancels an edit. edit_selected/delete_selected/undo_last, lines 673-730; refresh_buttons, lines 461-503.\n\n[verified] Numbering: a 'Next rally #' field lets you resume/insert anywhere; auto-advances to the next FREE number after each save and refuses a duplicate (planned_next_number/next_free_from, lines 371-393; save_rally, lines 656-664).\n\n[verified] Unsaved-work guard (v1.6.4): once a rally is fully marked (START+END) but not saved, both 'Mark START' (new rally) and 'Edit selected' are REFUSED with a prompt to Save Rally or Undo last first, so golden data is never silently lost (is_armed, lines 365-367; mark_start, lines 611-615; edit_selected, lines 674-678).\n\n[verified] Keys/buttons: this is a CLICK-driven dialog - there are NO keyboard hotkeys for marking; all interaction is buttons, dropdowns, and text fields. Global S/E/1-6/U hotkeys are explicitly ROADMAP ONLY, listed for a future python-vlc/Tk or HTML5 front-end (README.md:108-113). [design] So Studio docs must say 'click the buttons', not 'press S/E'. \n\n[verified] Saving: rows are appended live to the CSV as you go; the whole CSV is rewritten atomically (temp file + os.rename, with a Windows .bak rollback) so the live file is never lost on a failed write, and any extra/richer columns are preserved verbatim on rewrite (save_all, lines 310-335; load_rows, lines 273-304).\n\n[verified] Resume: labels persist to the CSV next to the video; re-open the SAME video and enable the extension and it reloads existing rallies and continues numbering. Recommended flow: open the video FIRST, then enable the extension. If enabled before the video loads it starts on a home-dir fallback path and LAZILY adopts <video>.rallies.csv on the first Mark START/Save (so nothing is mistakenly written to the home fallback); 'Refresh' re-points to whatever video is playing now. Playback POSITION is not restored - scrub to where you stopped. README.md:60-66; adopt_current_video_csv, lines 580-591; sync_to_current_video, lines 542-572.\n\n[verified] In-dialog Help button renders usage + the ending-reason decision guide INSIDE the dialog (VLC's own About cannot be set by an extension); show_help toggles a dedicated HTML panel, lines 758-771; HELP_HTML, lines 139-168.",

"output_csv": "[verified] EXACT schema: 6 columns, one row per rally, header written once when the file is created. HEADER constant vlc/rally_annotator.lua:128; docs/CSV_FORMAT.md:1-20; README.md:13-19.\n\nHeader line:\n rally_number,start_time,end_time,ending_reason,sport,shots_count\n\nExample rows (docs/CSV_FORMAT.md:6-10):\n 1,8.800,11.500,winner,badminton,9\n 2,24.389,46.589,unforced_error,badminton,21\n 3,49.183,54.683,let,badminton,\n\nColumns:\n - rally_number : int, 1-based, monotonic; continues across re-opening the same CSV (no duplicate IDs).\n - start_time : float, rally start in DECIMAL SECONDS from video start. Written with %.3f (save_all, line 317).\n - end_time : float, rally end in decimal seconds, always > start_time.\n - ending_reason : enum, one of { unknown (default), winner, forced_error,

unforced_error, service_fault, let, other }. REASONS list lines 121-124; full decision guide docs/ENDING_REASONS.md.\n - sport : enum, one of { badminton, tennis, table_tennis, pickleball, padel }. SPORTS list lines 125-127.\n - shots_count : int, OPTIONAL shot/stroke count for the rally; BLANK when not entered (see row 3). Added in v1.6; older 5-column CSVs still load (column reads blank). docs/CSV_FORMAT.md:19; CHANGELOG.md:42-55.\n\n[verified] Row write format: string.format('%d,%.3f,%.3f,%s,%s,%s', n, s, e, reason, sport, shots), where shots is '' when nil; any preserved 'extra' columns are appended after column 6 (save_all, lines 315-319). Loader is forgiving: any line whose first field is an integer is a rally row; columns beyond 6 are preserved verbatim on rewrite (load_rows, lines 273-304).\n\n[verified] WHERE it is written: a file named <video-stem>.rallies.csv NEXT TO the video file (same directory). If the video path cannot be resolved (extension enabled before a video loaded), it FALLS BACK to the user's home dir as rally_labels.csv (USERPROFILE on Windows, else HOME), but then lazily switches to the per-video CSV on first Mark START. resolve_out_path, lines 253-267; README.md:57-58.\n\n[verified] Consumer guidance (docs/CSV_FORMAT.md:21-26): read by COLUMN NAME (header stable, order may evolve, extra columns may be added later); decimal seconds NOT mm:ss; rows appended live; Undo last truncates the last row (one level).",

"screenshots": [],
"repo_facts": "[verified] License: MIT (LICENSE file; Copyright (c) 2026 Avi Dullu; README.md:133-134). Commercial-clean, matching the indexer's MIT/Apache/BSD-only guardrail.\n\n[verified] Latest version: v1.6.4 (dated 2026-06-18). Confirmed in THREE kept-in-sync places: VERSION constant vlc/rally_annotator.lua:88; the dialog/title golden test/dialog_layout.snapshot:1 ('Rally Annotator v1.6.4'); and CHANGELOG.md:3. NOTE: there are NO git tags/releases (git tag -l is empty) - versioning is the VERSION constant + CHANGELOG only. HEAD commit d5d59f2 'v1.6.4: guard unsaved rallies from silent loss + flag edit mode on Mark buttons (#13)' on branch main.\n\n[verified] Supported sports (5; all net-separated racquet sports sharing one error taxonomy): badminton (the reference sport), tennis, table_tennis, pickleball, padel. SPORTS list vlc/rally_annotator.lua:125-127; README.md:74-86. ending_reason vocabulary (7 values incl. default): unknown, winner, forced_error, unforced_error, service_fault, let, other (README.md:85; docs/ENDING_REASONS.md).\n\n[verified] OS support: cross-platform via VLC - Windows, macOS, Linux (install paths for all three; current_media_path decodes both Windows file:///C:/ and POSIX file:/// URIs, percent-decodes, and normalizes Windows backslashes, lines 228-249). Requires VLC 3.0.x (NOT 4.0; README.md:35).\n\n[verified] Quality/Ci: GitHub Actions (.github/workflows/ci.yml) on every push + PR runs luac5.1 -p syntax check, then the full Lua 5.1 dialog suite (test/dialog_test.lua) including a layout-snapshot golden diff; status badge README.md:3. Test suite is 65 assertions as of v1.6.4 (CHANGELOG.md:14). It stubs VLC's API, loads the real extension, drives callbacks, asserts results; it CANNOT cover real Qt GUI rendering or real media playback (no headless VLC), validated separately by 'vlc -vv --file-logging' + a manual click-through (test/README.md:88-94).\n\n[verified] Repo shape: tiny + self-contained. Tracked files (git ls-files): .gitattributes, .github/workflows/ci.yml, .gitignore, CHANGELOG.md, LICENSE, README.md, docs/CSV_FORMAT.md, docs/ENDING_REASONS.md, test/README.md, test/dialog_layout.snapshot, test/dialog_test.lua, vlc/rally_annotator.lua. Public repo: https://github.com/avidullu/rally-annotator.\n\n[verified] Roadmap (README.md:108-114): live-test the v1.6 dialog in VLC across all five sports; optional per-sport reason presets/hotkeys; alternative front-ends (python-vlc + Tk/Qt with global S/E/1-6/U hotkeys, and a zero-install HTML5 <video> page exporting the same CSV); the shots_count column is already DONE (v1.6); server/receiver columns still TBD behind a toggle.",

"file_refs": [
"C:\\Users\\avidu\\Projects\\rally-annotator\\README.md",
"C:\\Users\\avidu\\Projects\\rally-annotator\\vlc\\rally_annotator.lua",
"C:\\Users\\avidu\\Projects\\rally-annotator\\docs\\CSV_FORMAT.md",
"C:\\Users\\avidu\\Projects\\rally-annotator\\docs\\ENDING_REASONS.md",
"C:\\Users\\avidu\\Projects\\rally-annotator\\CHANGELOG.md",
"C:\\Users\\avidu\\Projects\\rally-annotator\\LICENSE",
"C:\\Users\\avidu\\Projects\\rally-annotator\\test\\README.md",
"C:\\Users\\avidu\\Projects\\rally-annotator\\test\\dialog_layout.snapshot",
"C:\\Users\\avidu\\Projects\\rally-annotator\\.github\\workflows\\ci.yml"
],
},
"trainLoop": {
"train_flow": "The annotate -> CSV -> `worker train` -> weights loop (engine repo, cwd=badminton-highlight-indexer):\n\n1. ANNOTATE: a human marks rally start/end + ending reason in the VLC `rally-annotator` Lua extension, which appends one row per rally and writes `<video>.rallies.csv` next to the video (rally_annotator.lua:128 HEADER, :258 filename).\n\n2. STAGE TO BUCKET/CORPUS: a corpus prefix is populated with `labels/` (the `.rallies.csv` files)

and, for real trajectories, `videos/` (the byte-identical videos the labels were made on) + `manifest.json`. Bucket layout `videos/ labels/ manifest.json weights/ outputs/<video\_id>/` (SETUP_NEW_MACHINE.md:242; 04-BUILD-AND-TEST-PLAN.md:57-66). The MD5-keyed join is the contract (02-COMPUTE-MAP-AND-CONTRACT.md:48-60).
 .\n\n3. `worker train` (backend/worker/`\_\_main\_\_.py::cmd\_train`, line 231): pull labels (and videos if --trajectory-source detector) to scratch -> build a per-video trajectory CSV -> assemble manifest.json -> SHELL OUT (subprocess, `\_\_main\_\_.py:321-328`) to `python -m training.gen0.harness` (backend must NOT import training ? the one-way wall, training/README.md:9-11) -> push the versioned bundle to `weights/<version>/` (`\_\_main\_\_.py:333-340`). Needs >=2 labeled videos for leave-one-video-out (`\_\_main\_\_.py:258`).
 .\n\nTwo trajectory sources, identical downstream (`\_\_main\_\_.py:236-241, 368-371`):
 .\n- synth (DEFAULT, GPU-free): `synth\_trajectory\_from\_labels()` (stub_runner.py:124) reads the label CSV and emits a deterministic Frame,Visibility,X,Y trajectory whose in-flight frames cover each labeled rally ? the trajectory ENCODES the labels. CPU-only; validates plumbing, NOT a production-meaningful model (stub_runner.py:131-133).
 .\n- detector: pull each video, resolve the real DetectorRunner (native WASB on GPU / stub for GPU-free wiring test), ffprobe fps+width (won't guess, skips on probe miss ? `\_\_main\_\_.py:293-296`), run_predict -> real Frame,Visibility,X,Y CSV (`\_\_main\_\_.py:298`).
 .\n\n4. HARNESS (training/gen0/harness.py): for each spec parse trajectory (TrackNetRunner.parse_trajectory_csv) + load GT (load_gt_intervals) -> build frame features (backend/features/rally_seq, train==serve) -> honest leave-one-video-out eval (lovo_run) -> fit FINAL LogisticRegression on all videos, tune threshold train-side (train_final, harness.py:78-95) -> run the ship-gate pre-flight (gate_verdict, harness.py:98) -> emit the ARTIFACT BUNDLE = `

"engine_label_schema": "Engine label-CSV schema (what the TRAIN path actually parses) is intentionally LOOSE and read by COLUMN NAME, via TWO readers depending on the code path:
 .\n\nA. The canonical reader `backend/eval/gt\_loader.py::load\_gt\_intervals` (used by the harness for GT, harness.py:88,180) accepts EITHER convention:
 .\n - header naming `start`/`end` OR `start\_time`/`end\_time` (gt_loader.py:30-31, _START_NAMES/_END_NAMES); ALL other columns ignored (gt_loader.py:16-18). It returns only (start, end) interval tuples.
 .\n - OR headerless positional columns 0,1 = start,end (gt_loader.py:70).
 .\n - Timestamps: decimal seconds OR MM:SS / HH:MM:SS (parse_timestamp, gt_loader.py:18). Blank lines and `#` comments ignored. strict=True (corpus/gate) raises file:line on bad/end<=start/negative rows; strict=False (harness uses strict=False, harness.py:88) skips+logs. So the ONLY required, load-bearing columns are start_time + end_time (or positional col0,col1).
 .\n\nB. The synth trajectory builder `synth\_trajectory\_from\_labels` uses `csv.DictReader` and reads literally `row["start\_time"]` and `row["end\_time"]` (stub_runner.py:139) ? so for the DEFAULT synth path the CSV MUST have a header with exactly `start\_time` and `end\_time` (the `start`/`end` short alias is NOT accepted here, only by gt_loader). Rows missing those keys are silently skipped (stub_runner.py:140-141).
 .\n\nThe worker's OWN inference output CSV (`\_write\_intervals\_csv`, `\_\_main\_\_.py:62-74`) writes header `start,end,shot\_count,ending\_reason` ? explicitly described in-code as "same schema as the rally-annotator labels" (`\_\_main\_\_.py:63-64`). The harness manifest expects per-video dicts `{name, trajectory, fps, frame\_width, labels}` (harness.py:213-214; built at `\_\_main\_\_.py:302-303,308-309`). The model bundle's own FEATURE_SCHEMA is from backend/features/rally_seq (FEATURE_SCHEMA/FEAT_NAMES, harness.py:42) ? independent of the label CSV columns.",

"schema_match": "YES ? the rally-annotator output CSV MATCHES the engine label schema with NO transform needed for the default synth train path.
 .\n\nrally-annotator writes (rally_annotator.lua:12,128; rally-annotator/docs/CSV_FORMAT.md:6):
 .\n Header: `rally\_number,start\_time,end\_time,ending\_reason,sport,shots\_count`
 .\n e.g. `1,8.800,11.500,winner,badminton,9`
 .\n\nEngine needs: a header containing `start\_time` + `end\_time` (decimal seconds, end>start). The annotator's header literally contains both column names by exact spelling, so:
 .\n- DEFAULT synth path (DictReader on row["start_time"]/row["end_time"], stub_runner.py:139): MATCHES exactly. The extra columns rally_number/ending_reason/sport/shots_count are simply not read. NO transform.
 .\n- gt_loader path (header-mapped): MATCHES; ignores all non-start/end columns (gt_loader.py:16-18). NO transform.
 .\n\nPrecise notes / non-gaps:
 .\n- Column ORDER differs from the worker's own intervals.csv (`start,end,...`) but order does NOT matter ? both readers map by name (CSV_FORMAT.md:24-25 explicitly: read by column name).
 .\n- The naming differs (annotator: start_time/end_time; worker self-output: start/end) but gt_loader accepts BOTH aliases, and the synth builder needs the *_time form which is exactly what the annotator emits ? so the annotator file is the safer of the two.
 .\n- shots_count may be BLANK (CSV_FORMAT.md:19) ? irrelevant, that column is unused by training.
 .\n- utf-8-sig / BOM tolerated (gt_loader.py:60). Blank lines / `#`

comments ignored.\n\nThe ONE real prerequisite is NOT a schema gap but an IDENTITY/JOIN gap: training keys video<->label by file MD5 and matches `labels/<name>.rallies.csv` to `videos/<name>.<ext>` BY STEM (___main__.py:271-275,280). So (a) the corpus filename stems must agree, and (b) the video bytes must be byte-identical to what was annotated (any re-encode changes the MD5 and breaks the join; collector export manifest currently omits an `md5` field ? flagged as an M2 prerequisite, 02-COMPUTE-MAP-AND-CONTRACT.md:62-68).",

"commands": [

"Annotate (rally-annotator, VLC): open the video in VLC with the rally_annotator.lua extension -> mark each rally start/end + ending reason -> writes <video>.rallies.csv next to the video (header rally_number,start_time,end_time,ending_reason,sport,shots_count). NEEDS: VLC 3.x + the Lua extension; a human; no compute.",

"Set up the box (SETUP_NEW_MACHINE.md): provision Ubuntu, get code on (git clone / git archive+scp), `./deploy/digitalocean/bootstrap.sh` (CPU) or `--gpu` (CUDA torch), `pip install -e .[dev]`, verify with `python -m pytest -q` (expect ~922 passed / 11 skipped). NEEDS: Python 3.8+, ffmpeg.",

"Train DIRECTLY via the harness (local manifest; CPU, minutes): `python -m training.gen0.harness --manifest <manifest.json> --floor eval\_baselines/heuristic\_lovo\_n6.json --out artifacts/rally\_tal\_gen0 --version 0.1.0`. manifest.json = list of {name,trajectory,fps,frame_width,labels}. NEEDS: a labeled corpus already on disk + per-video trajectory CSVs; numpy/scipy (training/requirements-train.txt). NEEDS >=2 videos (LOVO).",

"Train FROM A BUCKET, synth trajectories (DEFAULT, GPU-FREE, CPU mock): `python -m backend.worker train --corpus-uri gcs://<bucket> --out-uri gcs://<bucket> --version 0.1.0` (optionally `--floor <uri|path>`, `--trajectory-source synth` is the default, `--fps 30 --frame-width 1280`). Pulls labels/, synthesizes label-consistent trajectories, shells out to the harness, pushes weights/<version>/{weights.json,manifest.json}. NEEDS: a corpus prefix with labels/ (>=2 .csv); storage extra (`pip install -e .[storage-s3]` or `[storage-gcs]`) + creds; no GPU.",

"Train FROM A BUCKET, REAL trajectories (M3.5 path): `python -m backend.worker train --corpus-uri gcs://<bucket> --out-uri gcs://<bucket> --version 0.1.0 --trajectory-source detector --segmenter wasb\_hybrid` with `indexing.detector\_impl` = native (GPU box, native WASB) | stub (GPU-free wiring test) | auto (WSL on Windows). Pulls videos/ too, ffprobes fps/width, runs the resolved DetectorRunner. NEEDS: videos/ in the corpus (stem-matched to labels/), and for a REAL run a GPU + native WASB env (gpu_setup.sh) with wasb_badminton_best.pth.tar.",

"Inference (not training, for completeness): `python -m backend.worker infer --video-uri <uri> --out-uri <uri> --segmenter wasb\_hybrid --set indexing.skip\_ai\_handoff=true` (secret-free CPU smoke adds `RALLY\_DETECTOR\_IMPL=stub` + `env -u GEMINI\_API\_KEY`)."]

"gating": "What is REAL today vs blocked:\n\nREAL TODAY (verified):\n- The full annotate -> CSV -> `worker train` (synth) -> harness -> versioned weight bundle loop runs end-to-end GPU-FREE on a CPU box. SETUP_NEW_MACHINE.md verification log (2026-06-20, DO 2 vCPU, NO GPU): bootstrap OK, suite 922 passed/11 skipped, secret-free CPU inference detected 2 rallies, S3 (real DO Spaces sgp1) + GCS + MinIO worker round-trips passed. `worker train` synth orchestration + the detector-source-via-stub wiring are unit-tested (tests/test_worker.py:149,194). The model produced on the synth path is plumbing-meaningful ONLY, not production (stub_runner.py:131-133).\n\nGPU NEEDED FOR A REAL MODEL:\n- The Gen-0 head itself is tiny (numpy/scipy, ~1M params, ~4.5 GB VRAM, trains in minutes ? 02-COMPUTE-MAP-AND-CONTRACT.md:16) and runs on CPU. The GPU requirement is for REAL shuttle TRAJECTORIES (the training input), not the classifier.\n\nBLOCKED / IN-FLIGHT (the gate to a real-trajectory model):\n- Native WASB runner (M3.5) is IN-FLIGHT, not landed. The `LinuxNativeRunner` behind the DetectorRunner ABC does not exist yet / has never run (02:15; 04-BUILD-AND-TEST-PLAN.md:98-113). Until it lands, real `--trajectory-source detector` only works via WSL on Windows (the existing GPU+WSL path) or the GPU-free `stub` wiring test. The GPU box prep (mount_scratch.sh, gpu_setup.sh) is staged (SETUP_NEW_MACHINE.md:259-289) but WASB needs wasb_badminton_best.pth.tar present.\n- SHIP-GATE forces ship=False regardless of F1 (harness.py:133): (1) WASB checkpoint provenance C3 UNRESOLVED -> commercial ship refused; (2) F1@tIoU target NOT YET CALIBRATED (provisional, issue #172). Floor (heuristic LOVO ~0.480) + a paired sign test (n is small -> often not significant) must also pass. The product side owns the judge (backend/eval/regression.py); no model admits itself.\n\nLICENSING GUARDRAIL (hard): Gemini/paid LLMs are INFERENCE/serving/fallback ONLY, NEVER a training data source (terms/copyright). The bundle attests `no\_paid\_llm\_training\_labels: True` (harness.py:192). Label source must be human (rally-annotator golden CSVs, harness.py:178). Commercial-clean deps only (numpy/scipy BSD, repo-internal). Exclude minors; per-user data needs explicit opt-in.",

"file_refs": [

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker__main__.py (cmd_train line 231; _write_intervals_csv line 62; synth branch 304-309; detector branch

```

283-303; harness shell-out 321-328; push 333-340)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\training\\gen0\\harness.py
(train_final:78; gate_verdict:98; bundle emit 154-199; manifest arg 213)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\training\\README.md (the
train/serve wall + artifact-bundle boundary)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py (THE
label CSV reader; start/start_time + end/end_time accepted, line 30-31; strict modes 44-90)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\stub_runner.py (synth_trajectory_from_labels:124 ?
DictReader row['start_time']/['end_time'] at :139; StubDetectorRunner CPU mock)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\SETUP_NEW_MACHINE.md
(runbook; $7 bucket train/infer; $8 Gen-0 train; $9 GPU/native WASB; verification log
appendix)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\02-COMPUTE-MAP-AND-CONTRACT.md (the one data contract; MD5
join; train/serve wall)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\04-BUILD-AND-TEST-PLAN.md (milestone ladder; M2 train-from-
bucket; M3.5 native WASB parity; bucket layout)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_worker.py (cmd_train
synth orchestration :149; detector-source-via-stub :194; native-on-tracknet clean-fail :241)",
    "C:\\Users\\avidu\\Projects\\rally-annotator\\docs\\CSV_FORMAT.md (annotator output
schema)",
    "C:\\Users\\avidu\\Projects\\rally-annotator\\vlc\\rally_annotator.lua (HEADER line 128;
<video>.rallies.csv filename line 258)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\heuristic_lovo_n6.json (the --floor baseline)"
]
},
"uiSurface": {
    "ui_surface": "Three distinct khelsutra surfaces exist; they are NOT one app and deploy via
two different CI jobs (.github/workflows/test.yml:6-35).\\n\\n1) MARKETING SITE ? `site/public/`
(plain HTML, no build, no framework). Live at khelsutra.guru on Cloudflare (Worker + D1) AND a
Node+SQLite droplet. Pages today: `index.html` (the landing page, 583 lines, full inline brand
CSS) and `capture.html` (a standalone 1-page sub-page, the precedent for adding more marketing
sub-pages). Routing is path-based and dead simple: the Node host maps `/capture ? capture.html`
at `site/server.js:48-50`; the Worker serves everything non-/api via `env.ASSETS.fetch(request)`
at `site/src/index.js:84`. So a NEW static page is literally a new HTML file in `site/public/` +
(optionally) a clean-URL alias line in `site/server.js`. Brand tokens live inline in
`site/public/index.html:25-32` (--ink #0C261F, --green #19B36B, --lime #C7F2B0, --paper #F6F8F4;
font Plus Jakarta Sans). [verified]\\n\\n2) STUDIO SPA CONSOLE ? `web/` (Vite + React 19 + TS +
Tailwind v4). This is the operator console that DRIVES the engine (upload ? index ? pick rallies
? reel ? download) per `web/README.md:1-6`. It is a SCAFFOLD ONLY today: `web/src/App.tsx` is a
single static screen (33 lines) labeled `"operator console · scaffold (M1)\\n"; there is NO router
and react-router is NOT a dependency (`web/package.json:14-17` lists only react/react-dom) ?
grep for Routes/createBrowserRouter in web/src returns nothing. The picker screens land in M2+
per the tracked doc. Brand tokens are mirrored as Tailwind theme vars in
`web/src/index.css:5-11`. Same-origin `/api/*` calls proxy to the engine on :8000.
[verified]\\n\\n3) APPLIANCE DOC ? referenced as `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` BUT THIS
FILE DOES NOT EXIST in the khelsutra repo (Read failed; Glob for *APPLIANCE* returns nothing).
Per `docs/README.md:10-13`, the canonical control-plane/hosting docs (UI_PROPOSAL.md,
HOSTING_PLAN.md) currently live in the ENGINE repo's docs/, migrating to khelsutra `"when the
control plane lands." The only khelsutra console design doc that exists is
`docs/PER_RALLY_SELECTION.md` (the RallyPicker, owner-APPROVED 2026-06-20, MVP P1-P5 in
progress). So any cross-reference to a LOCAL_APPLIANCE_ARCHITECTURE.md must be created or re-
pointed ? do not cite it as existing. [verified]\\n\\n4) The public repo to link is `rally-annotator`
(https://github.com/avidullu/rally-annotator) ? a real, shipped, MIT-licensed VLC Lua plugin
(`vlc/rally_annotator.lua`) that hand-labels rally start/end + ending-reason + shot count to a
CSV (`rally-annotator/README.md`, `docs/CSV_FORMAT.md`, `docs/ENDING_REASONS.md`). [verified]",
    "page_placement": "Recommendation: BOTH surfaces, but split by job-to-be-done, and frame the
USP HONESTLY because the headline (`"annotate ? train your OWN model\\n") is only PARTLY real
today.\\n\\nHONESTY GROUNDING (must drive the copy ? do NOT invent capabilities):\\n- [verified]
REAL today = the ANNOTATE half. rally-annotator is a shipped, MIT, public VLC plugin producing
golden-label CSVs (`rally-annotator/README.md`). The engine ingests those CSVs directly (`rally-
annotator/docs/CSV_FORMAT.md:36-42` shows `python -m backend.eval.calibrate_wasb --labels ...`

```

and `curate import-local`).\n- [verified] PARTLY real = the TRAIN half. The engine has a Gen-0 train-from-bucket CLI (`badminton-highlight-indexer/backend/worker/\_\_\_main\_\_.py:232-318`, commit d5407eb), but TODAY it runs on a CPU-mock/`synth` trajectory (`--trajectory-source synth` is the default; `detector` path is the `M3.5 first real training` path, `\_\_\_main\_\_.py:236-240`). It trains a small ~1M-param TAL head, NOT a VLM. It is a developer/CLI path with NO Studio UI.\n- [design]/[researched] ROADMAP = the conversational VLM `train your own model` product. Per `badminton-highlight-indexer/docs/OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md` (Status: PLAN; `No owned-model implementation starts until the owner greenlights`): base = Qwen3-VL, framework = ms-swift, `head now ? VLM later`, and explicitly `The moat = the consented dataset + the eval harness, not the weights.`\n- [verified] The marketing site already commits to this exact honesty discipline ? see the `Read this honestly` boxes at `site/public/index.html:351-354` and the `What KhelSutra is not` list `:400-415`. New pages MUST match that voice.\n\nSo the moat is `your consented, hand-labeled corpus makes YOUR detector sharper` ? NOT `click a button to train a model today.` Position accordingly.\n\nPLACEMENT:\nA) MARKETING (educate/convert) ? site/public/, public-facing. ADD ONE new static page, e.g. `site/public/own-model.html` (or `/annotate`), styled like `capture.html`. Purpose: tell the `annotate your footage ? it trains your own detector` story as the bring-your-own-model differentiator, link the public rally-annotator GitHub repo, embed screenshots, and give step-by-step VLC setup + CSV-output explanation (all sourced from rally-annotator/README.md install steps). Tag every claim: annotator = [verified/available now], `trains your own model` = [roadmap, powered by your contributed labels], VLM/conversational = [roadmap]. Cross-link from the landing nav (`index.html:218-223`) and the `gets sharper from real academy footage` section (`index.html:327-356`) which is the natural anchor for this USP, plus the footer (`index.html:521-531`, mirroring the existing /capture link). REASON: marketing is the convert surface, it's a trivial new-HTML-file add (capture.html proves the pattern), and the USP is fundamentally an EDUCATION/positioning story (most viewers will never run a CLI).\n\nB) STUDIO SPA (operate) ? web/, behind the invite-only app. ADD an in-app `Improve / Train` route, but scoped HONESTLY to what exists: a correction/labeling loop on detected rallies (accept/reject/nudge) that BUILDS the consented corpus ? this is exactly the deferred P10 `PATCH /api/segments correction loop ? A4 Rally Review` in `docs/PER\_RALLY\_SELECTION.md:122` (gated). Do NOT put a `Train my model now` button in the SPA ? no user-facing training endpoint exists; training is the offline `backend/worker train` CLI. The SPA's honest contribution is `label/correct ? grow your corpus,` with a link out to the rally-annotator repo for bulk offline labeling. REASON: the SPA is the OPERATE surface where a coach actually produces labels; but it must stay tethered to real endpoints (the whole PER_RALLY_SELECTION MVP rides existing /api/query+compile with zero new intelligence) ? so the train page here is a [design]/gated screen, not MVP.\n\nNET: marketing page = ship now (educates + converts, links the live public repo). SPA train/correct screen = design-tracked + owner-gated (it depends on the unbuilt PATCH /api/segments loop). The annotate SETUP/HELP content (VLC install, CSV format, ending-reason guide) belongs on the marketing page since the annotator is a standalone external tool, with the SPA linking to it rather than duplicating it.",

"routing_reuse": "MARKETING SITE (site/) ? net-new, low-risk, ship now:\n- ADD `site/public/own-model.html` (or `byo-model.html`). Reuse the standalone-sub-page pattern of `site/public/capture.html` verbatim: same `` (favicon data-URI, Plus Jakarta Sans preconnect+link, inline :root brand tokens at `capture.html:13`), same `.sheet/.head/.brand/.step/.cols(do/dont)` CSS so it matches without a build step. Reuse the brand palette from `site/public/index.html:25-32`.\n- ADD a clean-URL alias in `site/server.js:48-50` (one line: `: pathname === `"/own-model" ? `own-model.html``). The Cloudflare Worker needs no change ? `env.ASSETS.fetch` at `site/src/index.js:84` already serves any file in public/.\n- ADD nav + footer links in `site/public/index.html` nav (`:218-223`) and footer (`:528`, next to the existing `/capture.html` link), and a CTA inside the `it gets sharper from real academy footage` section (`:327-356`).\n- Reuse existing honesty components: the `.honest` callout block style (`index.html:152-154`, used at `:351-354`) and the `.notes` `what it is not` list (`:156-158`) to fence roadmap-vs-real.\n- Screenshots: store under `site/public/` (e.g. `site/public/img/annotate-\*.png`); the precedent is `site/og.png` + the generator `site/make\_og.py`.\n\nSTUDIO SPA (web/) ? net-new, but introduces routing (currently NONE) and is owner-gated:\n- ADD a client router. Today `web/src/App.tsx` is a single static screen and react-router is NOT installed (`web/package.json:14-17`). To add a `Train/Improve` route you must add a router dep (react-router-dom) or a minimal hash switch. Keep it tiny; CI gates on `npm run typecheck && test && build` (`github/workflows/test.yml:23-35`).\n- ADD components for the correction/label loop: a RallyReview screen (accept/reject/nudge per detected rally) + a `Contribute to your corpus` panel. This is the P10 row already specified in `docs/PER\_RALLY\_SELECTION.md:122` (`PATCH /api/segments/{id} corpus loop (PR-4/B6) ? A4 Rally Review`) and its data plumbing (`candidate\_segments.human\_label/notes/confirmed\_segment\_id`) exists-but-is-dead per `PER\_RALLY\_SELECTION.md:52`. So this depends on a NEW engine endpoint

(`PATCH /api/segments/{id}`, listed `"new"` in that doc's §4.3 table) ? owner-gated, do not build without sign-off.\n- Reuse: the typed engine client `web/src/api/client.ts` + types `web/src/api/types.ts` (extend with a segment-patch call) and `web/src/lib/log.ts` correlation-id logging; reuse the brand Tailwind utilities from `web/src/index.css:5-11` (bg-ink/text-lime/text-green) and the existing App shell header (`web/src/App.tsx:4-11`).\n- Do NOT add a `"train now"` button or endpoint ? training is the offline CLI `backend/worker train` (`badminton-highlight-indexer/backend/worker/\_\_main\_\_.py:232`), not a Studio action.\n\nASSETS TO REUSE (summary): brand CSS tokens (`site/public/index.html:25-32` / `web/src/index.css:5-11`), the capture.html sub-page skeleton, the `.honest`/`.notes` honesty components, the SPA scaffold + typed API client + logger, and the live public rally-annotator README content for the setup/help copy (do not re-author the VLC instructions ? link + lightly summarize).",

"doc_convention": "Create a tracked project doc in the khelsutra repo following `badminton-highlight-indexer/docs/PROJECT\_DOC\_TEMPLATE.md`, mirroring the discipline of the existing `khelsutra/docs/PER\_RALLY\_SELECTION.md` (its direct peer). Required structure:\n- Status header line: `Status: DRAFT ? owner-gated · Owner: Avi · Created/Last-updated: <date>`, Lifecycle `DRAFT ? IN PROGRESS ? DONE ? archived`, Tracking-anchors (the §7 tracker is source of truth, index it in `docs/README.md`, pointer in `NEXT\_STEPS.md` once work starts), Relation line (peer of PER_RALLY_SELECTION.md), and the Honesty note.\n- Sections: 0 TL;DR · 1 Problem & goal · 2 Decisions locked (table) · 3 Foundation (the licensing/roadmap facts) · 4 Design/architecture + REUSE MAP · 5 Threat model (privacy: minors-excluded, opt-in only) · 6 Honest limits · 7 Deliverables & progress tracker (ONE PR per row, branched from origin/master, no stacking) · 8 Open questions (split blocking-gates) · 9 Definition of done · 10 References + Changelog.\n\nHOUSE VOICE / HONESTY RULES (non-negotiable, from the codebase's own conventions):\n- Tag EVERY engine-fact claim `[verified]` (checked against code this session) / `[design]` (proposed) / `[researched]` (needs sign-off) ? exactly as `PER\_RALLY\_SELECTION.md:7` and `KHELSUTRA\_PRODUCT\_OVERVIEW.md` do.\n- Match the marketing site's existing `"Read this honestly"` tone (`site/public/index.html:351-354`): say plainly that `"gets sharper / train your own model"` is the model being built toward (powered by contributed, consented footage), NOT a switch that is on today.\n- Carry the committed guardrails verbatim from CLAUDE.md / OWNED_MODEL_IMPLEMENTATION_PLAN.md: minors excluded from any training pool; per-user training data needs separate explicit opt-in; never train on paid-LLM output; MIT/Apache/BSD only. The marketing copy already states the minors + opt-in promises (`index.html:342-349`) ? reuse that wording.\n- Per CLAUDE.md workflow: substantial work like this gets the tracked doc; branch off master, ONE PR per item, no PR stacking, do not open a PR unless asked, and (Launch-Report convention) any default-config/quality flip ships with a documented impact note. The SPA train/correct screen and any new engine endpoint stay OWNER-GATED.\n- Branching safety: branch from `origin/master` explicitly, re-verify HEAD before commit, stage explicit paths (shared-checkout collision risk noted in memory).",

"file_refs": [
 "C:\\Users\\avidu\\Projects\\khelsutra\\site\\public\\index.html",
 "C:\\Users\\avidu\\Projects\\khelsutra\\site\\public\\capture.html",
 "C:\\Users\\avidu\\Projects\\khelsutra\\site\\server.js",
 "C:\\Users\\avidu\\Projects\\khelsutra\\site\\src\\index.js",
 "C:\\Users\\avidu\\Projects\\khelsutra\\web\\README.md",
 "C:\\Users\\avidu\\Projects\\khelsutra\\web\\src\\App.tsx",
 "C:\\Users\\avidu\\Projects\\khelsutra\\web\\src\\index.css",
 "C:\\Users\\avidu\\Projects\\khelsutra\\web\\src\\api\\client.ts",
 "C:\\Users\\avidu\\Projects\\khelsutra\\web\\src\\api\\types.ts",
 "C:\\Users\\avidu\\Projects\\khelsutra\\web\\src\\lib\\log.ts",
 "C:\\Users\\avidu\\Projects\\khelsutra\\web\\package.json",
 "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\README.md",
 "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PER_RALLY_SELECTION.md",
 "C:\\Users\\avidu\\Projects\\khelsutra\\NEXT_STEPS.md",
 "C:\\Users\\avidu\\Projects\\khelsutra\\.github\\workflows\\test.yml",
 "C:\\Users\\avidu\\Projects\\rally-annotator\\README.md",
 "C:\\Users\\avidu\\Projects\\rally-annotator\\docs\\CSV_FORMAT.md",
 "C:\\Users\\avidu\\Projects\\rally-annotator\\docs\\ENDING_REASONS.md",
 "C:\\Users\\avidu\\Projects\\rally-annotator\\vlc\\rally_annotator.lua",
 "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md",
 "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\KHELSUTRA_PRODUCT_OVERVIEW.md",
 "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\OWNED_MODEL_IMPLEMENTATION_PLAN.md",
 "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker__main__.py"
]


```

]
},
"usp": {
  "setup_help": [
    "STEP 0 Prerequisites [verified]: VLC 3.0.x only, not 4.0 (plugin targets 3.x; rally-annotator README). A match video shot per the capture checklist. For TRAIN, a GPU box with native-WASB, or the CPU-mock path for a dry run.",
    "STEP 1 Install the VLC plugin [verified, README]: copy the Lua file into VLC's Lua extensions folder (Windows APPDATA vlc lua extensions; macOS Library Application Support org.videolan.vlc lua extensions; Linux .local share vlc lua extensions), reload extensions, then enable Rally Annotator from the View menu.",
    "STEP 2 Record [verified]: reuse the existing one-page capture checklist (capture.html) - fixed tripod, full court, landscape, whole session. Link to it, do not duplicate.",
    "STEP 3 Annotate [verified, README Use]: open the video in VLC first, enable the extension, pick Sport, then per rally Mark START at the serve-contact frame, Mark END at the shuttle-dead frame, choose Ending reason (defaults to unknown), optional Number of shots, Save Rally. Edit/Delete/Undo to fix. Boundary discipline is the quality lever.",
    "STEP 4 Export CSV [verified, CSV_FORMAT.md]: the plugin auto-writes a .rallies.csv next to the video with columns rally_number,start_time,end_time,ending_reason,sport,shots_count (decimal seconds). This is the lighter schema; the richer Roora golden-set schema is the vendor spec, not plugin output - keep distinct.",
    "STEP 5 Train [verified path; owner-gated as a product]: register labels via python -m src.cli.curate import-local, then python -m backend.worker train. Default trajectory-source synth is a CPU-mock (no GPU); trajectory-source detector is real WASB and GPU/native-WASB-gated. Needs at least 2 labeled videos. Present as open pipeline in active build, not a Studio button.",
    "STEP 6 Use the model [researched/design]: train pushes a versioned weights bundle to the bucket; per the plan, if Gen-0 clears the ship-gate A/B it becomes the rally/highlight extractor driving the existing record-upload-reel loop. Per-user serving UI is design, not shipped."
  ],
  "screenshot_plan": [
    "CARDINAL RULE: the rally-annotator repo contains NO committed images [verified: only LICENSE, README, CHANGELOG, docs markdown, the Lua file, tests; zero image files]. There are NO existing annotator screenshots to reuse; every screenshot is TO-CAPTURE. Do NOT fabricate or AI-generate any UI screenshot.",
    "TO-CAPTURE 1 install: the VLC extensions folder with the Lua file in it.",
    "TO-CAPTURE 2 hero: the VLC View menu Rally Annotator entry plus the open dialog (Mark START, Mark END, Ending reason, Save Rally, Recent rallies). Load in VLC 3.0.x and screenshot.",
    "TO-CAPTURE 3 boundary accuracy: VLC paused on a serve-contact frame with the Start field just filled; use a consented no-minors clip and redact faces.",
    "TO-CAPTURE 4 output: the .rallies.csv opened in a sheet showing the real columns.",
    "TO-CAPTURE 5 optional train: a terminal showing the worker train output on the CPU synth path.",
    "REPO-SOURCED text (embed verbatim, NOT screenshots): the CSV example and install commands from README and CSV_FORMAT.md; the ending-reason table from ENDING_REASONS.md.",
    "DO NOT: invent a polished Studio annotator screenshot, screenshot minors, or show a Train button in the web app. Use labeled screenshot-to-be-captured placeholders, never fabricated mockups."
  ],
  "honest_caveats": [
    "The annotator is a SEPARATE VLC desktop plugin, NOT an in-browser/in-Studio annotator [verified: single Lua extension for VLC 3.0.x]. Labeling happens in VLC; the page only links to and documents it.",
    "The Studio Train-your-own-model pages do NOT exist yet - net-new design [verified: site has only index.html, capture.html, og.png; zero mentions of GitHub/annotator/train]. Tag the deliverable design.",
    "TRAIN is GPU/native-WASB-gated for REAL trajectories [verified: worker train detector source resolves a DetectorRunner with a healthcheck]. The only no-GPU path is synth, a CPU mock for pipeline validation, not a production model.",
    "Train-your-own-model as a one-click product feature does NOT exist; it is a CLI, and the Gen-0 ship decision is owner-gated [verified: plan Decisions item 1 awaiting owner; Phase 1 not started].",
    "There are MANUAL/CLI steps throughout: install by hand, annotate per-rally in VLC, register labels with curate import-local, then invoke worker train. A power-user/contributor flow, not a polished funnel."
  ]
}

```

```

    "Two CSV schemas exist - the free annotator lighter schema vs the richer Roora vendor spec
[verified]. Do not present Roora-only fields as plugin output.",
    "No accuracy claim is supportable: there is NO benchmark of a user-trained model vs a paid
LLM. The moat is ownership/licensing/privacy/consent/minors-excluded, not measured accuracy.",
    "The owned base model (Qwen3-VL, TAL-head-first) is roadmap [researched:
OWNED_MODEL_IMPLEMENTATION_PLAN.md]; base pretraining provenance is unauditable and needs owner
risk-acceptance.",
    "Real constraint to state: VLC 3.0.x only (4.0 unsupported), install paths differ per OS."
  ],
  "file_refs": [
    "C:/Users/avidu/Projects/rally-annotator/README.md",
    "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
    "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
    "C:/Users/avidu/Projects/rally-annotator/LICENSE",
    "C:/Users/avidu/Projects/rally-annotator/vlc/rally_annotator.lua",
    "C:/Users/avidu/Projects/khelsutra/site/public/index.html",
    "C:/Users/avidu/Projects/khelsutra/site/public/capture.html",
    "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/ROORA_LABELING_GUIDELINES.md",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/KHELSUTRA_PRODUCT_OVERVIEW.md",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__main__.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/CLAUDE.md"
  ],
  "usp_framing": "POSITIONING (one line): \"Annotate your own footage and train a model that's
YOURS - on an open, readable stack you control.\" This is the bring-your-own-model moat,
defensible against paid LLMs on axes that map 1:1 to KhelSutra's EXISTING guardrails, so the USP
is credible rather than marketing spin.\n\nTHE CORE CONTRAST WITH PAID LLMs: You cannot train
Gemini/OpenAI/Anthropic on YOUR data. They are closed - you rent inference, never own weights,
and your corrections never compound into an asset you keep. KhelSutra's roadmap is the inverse:
a small, OWNED rally/highlight model where the moat is the consented dataset plus the eval
harness, not the weights [researched: OWNED_MODEL_IMPLEMENTATION_PLAN.md North Star]. The rally-
annotator is the front door to building that dataset. Frame it as OWNERSHIP plus DATA CONTROL,
NOT more-accurate-than-Gemini - there is no benchmark of a user-trained model vs a paid LLM, so
do NOT claim accuracy superiority. The honest claim: a model trained on YOUR courts, camera
angles, and busy multi-court halls, with full control of where the data lives.\n\nTIE EACH MOAT
AXIS TO A GUARDRAIL THAT ALREADY EXISTS: (1) Commercial-clean licensing [verified]: the
annotator is MIT (rally-annotator LICENSE); the owned-model framework and base target
MIT/Apache-2.0/BSD only, Qwen3-VL Apache primary [researched: CLAUDE.md]. (2) Paid-LLM-never-a-
training-source [verified: CLAUDE.md plus plan M0.3]: sell it as a feature - we never launder
paid-LLM output into the model, so the model you train stays clean of another vendor's terms and
copyright. (3) Privacy plus consent [verified: already on the live site index.html Your-data-
stays-yours plus product overview]: footage stays the user's; training use is explicit opt-in,
never automatic; nothing published. The annotator runs locally in the user's own VLC and writes
a CSV next to the video, so labeling happens without footage leaving the user's machine. (4)
Minors excluded [verified: CLAUDE.md plus Roora guidelines data-handling block plus live site
Minors-are-protected]: footage with minors never enters the training pool. Keep this prominent
because the audience is academies (children).\n\nHONEST GUARDRAIL ON THE FRAMING ITSELF: Today
the train path is a CLI Gen-0 harness that is owner-gated and not yet a product feature
[verified: worker main; plan Phase 1 Gen-0 ship decision still awaiting owner]. So the Studio
pages must present train-your-own-model as an OPEN, INSPECTABLE CAPABILITY plus DIRECTION (with
the public annotator as the real, usable first step), not as a one-click button live in the app.
The defensible-today claim: the data-collection tool is real, open-source (MIT), and yours; the
training pipeline is open and in active build.</usp_framing>\n</invoke>\n"
  }
}

```

Produce: the recommended dedicated pages (Annotate your footage · Train your own model · Setup & use) with placement (marketing site vs SPA console vs both); the grounded annotate?CSV?train?model flow; the EXACT schema bridge between the annotator CSV and the engine label CSV; a step-by-step setup runbook; an honest screenshot plan (real sources only, never fabricated); honest USP copy; honest limits; open decisions; a progress tracker; and the full design_markdown per the template. Be brutally honest: real training needs a GPU + native WASB (in-flight); the annotator is a separate VLC tool (not in-browser); paid LLMs are never a training source (licensing guardrail). Cite grounding refs inline.

Return the schema.

2. assistant (2026-06-20T10:12:47.585Z)

I'll work through this systematically. Let me first verify the key facts against the actual files, then produce the consolidated design doc.

3. user (2026-06-20T10:12:48.551Z)

```
1      <!--
2      PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
3
4      A "tracked project doc" is how this repo carries substantial design/project work: it can
be
5      ITERATED on, its progress TRACKED, and it is MARKED DONE against explicit deliverables,
then
6      archived. Standardizes the emergent pattern in docs/RALLY_QUALITY_RESEARCH.md §7 and
7      docs/GOLDEN_REGRESSION_FIXTURES.md.
8
9      Rules of thumb:
10     - §7 (the progress tracker) is the SOURCE OF TRUTH ? one row per independently-shippable
PR.
11     - Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs
sign-off),
12     or [design] (proposed). The doc reflects real shipped state, never aspirational.
13     - It POINTS to detail (code anchors, research artifacts), it does not duplicate it.
14     - Sections marked OPTIONAL are included only when relevant (e.g. Foundation/Threat-model
for
15     research- or security/privacy-heavy work). Delete sections that do not apply.
16     - Move to docs/archives/ once Status = DONE (per the archive policy: only terminal-state
work).
17     -->
18
19     # <Project / Plan Title>
20
21     > **Status:** `DRAFT` ? `<owner-gated? | in progress>` . **Owner:** `<name>` .
**Created:** `<YYYY-MM-DD>` . **Last updated:** `<YYYY-MM-DD>`
22     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
23     > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
24     > **Relation to existing docs:** `<extends | supersedes | peer-of ?>`
25     > **Honesty note:** mark each claim `[verified]` / `[researched]` / `[design]`. Not
legal/financial advice where applicable.
26
27     ---
28
29     ## 0. TL;DR
30     <2?5 sentences: the goal, the approach, and the single most important caveat. Call out
any finding that is already actionable on its own.>
31
32     ## 1. Problem & goal
33     <Why now; the concrete user-facing outcome; what "good" looks like. Link the code/docs a
reader must read to get current.>
34
35     ## 2. Decisions locked
36     <Table ? capture answered questions so they are not relitigated.>
37
38     | # | Decision | Source / date | Implication |
39     |---|-----|-----|-----|
40     | D1 | ? | ? | ? |
41
42     ## 3. Foundation ? research / prior art *(OPTIONAL)*
43     <Include when the plan rests on external facts: cited research, legal/compliance,
benchmarks, prior-art survey. Mark researched-vs-verified; list any sign-off gates this
```

```

creates.>
44
45     ## 4. Design / architecture
46     <The actual plan. Data contracts, diagrams, and a REUSE MAP ? name the existing
modules/code this builds on so it is clear what is new vs reused.>
47
48     ## 5. Threat model / risk table *(OPTIONAL ? security / privacy / compliance work)*
49     <Who/what is protected vs NOT; residual risks stated plainly.>
50
51     ## 6. Honest limits ? what this does NOT do
52     <The false-confidence guardrail: what a green suite / a DONE status does NOT prove.>
53
54     ## 7. Deliverables & progress tracker  ? **source of truth**
55     Legend: ? Todo . ? In progress . ? Done . ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
56
57     | ID | Deliverable | Depends on | Gated? | Status | PR |
58     |----|-----|-----|-----|-----|----|
59     | P0 | ? | ? | No | ? | ? |
60
61     ## 8. Open questions ? owner / external *(split blocking-gates from nice-to-knows; name
who decides)*
62     1. ?
63
64     ## 9. Definition of done
65     <Explicit acceptance checklist. When all ticked ? flip `Status ? DONE` and archive.>
66     - [ ] ?
67
68     ## 10. References
69     **Internal code anchors `[verified]`:** `<paths>`. **Internal docs:** `<docs>`.
**External:** `<cited sources>`.
70
71     ---
72
73     ### Changelog
74     - `<YYYY-MM-DD>` ? `<what changed>`.
75

```

4. user (2026-06-20T10:12:49.189Z)

```

1     # Interactive per-rally selection ? reel builder (RallyPicker)
2
3     > **Status:** `IN PROGRESS` ? design **APPROVED by owner 2026-06-20**; building the MVP
(P1?P5). Downstream milestones (multi-video auth, per-rally preview endpoint, any `/api/compile`
default change) remain owner-gated. . **Owner:** Avi . **Created:** 2026-06-20 . **Last
updated:** 2026-06-20
4     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6     > **Relation:** second major tracked work item in the `kheksutra` repo (peer of
`REQUEST_ACCESS_FORM.md`); mirrors the engine repo's `PROJECT_DOC_TEMPLATE` discipline.
Primarily realizes alpha screen **A5 Highlights** (engine repo `UI_PROPOSAL.md:47`, exec-plan
**M4** `UI_ALPHA_EXECUTION_PLAN.md:39`); its correction-loop extension realizes **A4 Rally
Review** (`UI_PROPOSAL.md:46`, exec-plan **M3** `:38`). Consumes the engine's `/api/query` +
`/api/compile` contract.
7     > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
code this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9     ---
10
11     ## 0. TL;DR
12     Build the **pick-rallies ? reel** surface as a **hybrid**: a selectable rally
**list/grid** is the workhorse; a **deterministic query bar** ("rallies over 10s, longest

```

first") pre-selects into the **same** selection set; a sticky footer compiles the selection into one downloadable MP4. The entire MVP runs on **existing engine endpoints** ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights` `[verified backend/main.py:331-340, 342-396, 307-328]` ? with **zero new engine intelligence and zero new endpoints**. The single most important caveat: queries and chips are scoped to **length / count / order / shot-count only**; shot-type, player, and score are **roadmap-only** (no detector, no column) and must not be faked. The biggest UX-vs-effort tension: **no per-rally preview is possible today** ? only the **final compiled reel** can stream ? so MVP selects on text metadata, and visual preview/timeline is a deliberate later milestone.

13

14 **## 1. Problem & goal**

15 Today the alpha can detect rallies and stitch a reel, but reel-building is either a **silent `stitching.min\_shot\_count` threshold** `[verified backend/main.py:362-383]` or a static checkbox list whose data plumbing is even broken ? the bundled static UI reads `data.rallies` while the API returns `play\_segments`, so it renders nothing `[verified backend/api/static/app.js:344 vs backend/main.py:340]`. The owner's framing is **dynamic per-video selection, not a fixed threshold**: a coach should say "give me the long rallies from this session" and get exactly those, then fine-tune by hand.

16

17 **The concrete outcome ("good")**: A coach uploads a 40-minute session; the engine finds 60 rallies. They type **"top 15 longest"**, eyeball the auto-selected cards, untick two warm-up rallies, name it **"Tuesday ? long points"**, and download a ~6-minute reel ? without learning any threshold knob.

18

19 **Goal**: the smallest **honest** surface that turns engine-detected rallies into a user-curated, downloadable reel, with both **fast** (query) and **precise** (checkbox) selection sharing **one** state.

20

21 **Non-goals (alpha)**: no conversational LLM (`UI\_PROPOSAL.md:66` defers it); no shot-type/player/score filters (no data exists); no per-rally video preview while it requires unbuilt streaming; no multi-tenant auth (single-box alpha).

22

23 **## 2. Decisions locked**

24 | # | Decision | Source / date | Implication |

25 |---|-----|-----|-----|

26 | D1 | Dominant surface = selectable rally **list/grid** | Synthesis 2026-06-20 (Grid scored 15, fit-to-data 4) | Scales to long matches via bulk+filter; shows honest fields only |

27 | D2 | Fast entry = **deterministic** client-side `parseQuery()` (no LLM) | `[verified UI\_PROPOSAL.md:66]` conversational QA deferred | Predictable, unit-testable, offline, zero API cost |

28 | D3 | **One** shared `Set<segment\_id>`, mutated by query + checkbox + (future) timeline | Synthesis 2026-06-20 | No second compile path can diverge from what the user sees |

29 | D4 | Compile via existing `POST /api/compile {video\_id, segment\_ids, output\_filename}` | `[verified backend/main.py:342-396]` | Selection order is harmless ? stitcher `\_merge\_overlapping` de-dupes `[verified compiler.py:80-105, 311-317]` |

30 | D5 | Query grammar (alpha) = length, count, order, **shot_count only** | `[verified database.py:122-135, gemini.py:470-485]` | Only fields the payload actually returns; grey out the rest |

31 | D6 | MVP backend = **zero new endpoints** | `[verified]` reuse map (§4) | Ships the full loop on the already-shipped contract |

32 | D7 | Replace the silent `min\_shot\_count` cut with **explicit user selection** | Owner framing + `[verified main.py:362-383]` | UI **warns** about the floor; it never hides why a rally vanished |

33 | D8 | **Default selection = ON** (curate-by-removal) | Owner 2026-06-20 | Rallies load all-selected (matches today's "all rallies" reel); user removes warm-ups. Footer/empty UX assumes a non-empty start |

34 | D9 | **Preview-before-select** is NOT a ship-blocker for the MVP | Owner 2026-06-20 | MVP selects on text metadata + previews the **final compiled reel**; the source/clip-stream endpoint (per-rally preview) is the next milestone (P9), not MVP |

35 | D10 | **MVP is job-tethered** (carry `result.video\_id`); no `GET /api/videos` | Owner 2026-06-20 | Smallest honest slice; multi-video home + identity scoping is P8 (gated) |

36 | D11 | `min\_shot\_count`: **warn, do NOT override** in the MVP | Owner 2026-06-20 | The UI surfaces the floor; an explicit-selection override of the compile default is deferred (would trigger the Launch Report convention) |

37

```

38     ## 3. Foundation ? what the engine gives us today `[verified this session]`
39     The whole design rests on the engine's real, already-shipped contract. Read against the
engine repo (`badminton-highlight-indexer`):
40
41     - **The rally store is `play_segments` ? columns `id, video_id, start_time, end_time,
duration, server, receiver, metadata` `[verified database.py:124-135]`. `id` is the integer the
UI selects and passes to compile; `start_time/end_time/duration` are REAL seconds (duration
computed at insert).
42     - **Listing rallies = `POST /api/query {video_id}` ? `{play_segments:[...]}` ? the
*only* rally-list path today; it JSON-parses `metadata` and joins `events` `[verified
main.py:331-340, database.py:427-475]`.
43     - **Building a reel = `POST /api/compile {video_id, segment_ids:[int],
output_filename}` ? loads `videos.filepath`, filters all segments to the requested
`segment_ids`, applies the `min_shot_count` floor, extracts `time_pairs=[(start,end)]`, and
calls `VideoCompiler.compile_highlights(raw_path, segments, name)` `[verified main.py:342-396,
compiler.py:303]`, which **merges overlaps** then per-clip re-encodes ? concat ? one MP4
`[verified compiler.py:311-317]`. Selection order/duplicates are therefore safe.
44     - **Downloading = `GET /api/highlights/{video_id}/{filename}` ? streams the compiled
MP4. Compile returns a **server-local filepath, NOT a URL** `[verified main.py:394]`, so the SPA
constructs this download URL itself. The compiled reel is the **only streamable video that
exists today** `[verified main.py:307-328]`.
45     - **The floor knob is readable** ? `GET /api/config` exposes `stitching.min_shot_count`
`[verified main.py:121-123]`, so the UI can warn before a selected rally is silently dropped.
46     - **Job entry** ? `GET /api/jobs/{job_id}` returns `result.video_id` `[verified
main.py:274-293]`, which is how the MVP reaches a video without a (nonexistent) list-videos
endpoint.
47
48     **Honest fields vs fabricated/roadmap (critical for the UI):**
49     - **Honest today** (default `default_segmenter: "gemini"`, per `config.json`
`[verified]`): `start_time`, `end_time`, `duration`, and `metadata.shot_count` (Gemini's
`shuttle_exchange_count`, fallback `max(3, int(dur/0.8))`) + `metadata.shot_count_confidence` (a
**string**, often "Unknown") `[verified gemini.py:470-485]`. On the gemini path
`server`/`receiver` are written `None` ("no fabricated data") `[verified gemini.py:480-487]`.
50     - **Fabricated ? do NOT surface:** the `motion` demo segmenter fills `server`/`receiver`
via `random.sample(...)` , a random `shot_count`, and random `events` (serve/smash/foul/winner)
`[verified motion.py:191-198]`. `query` returns these columns + `events`, but they are not
ground truth and must stay hidden.
51     - **Roadmap ? no column, no detector:** shot-type per shot, player identity, match/point
score, game/set grouping. None exist anywhere in the schema; do not consume them `[verified
database.py whole-schema 122-148]`.
52     - **Correction-loop columns exist but are dead:**
`candidate_segments.human_label/notes/confirmed_segment_id` are defined but no endpoint writes
them `[verified database.py:166-168]` ? that's the M4 corpus extension (engine PR-4/B6,
`UI_ALPHA_EXECUTION_PLAN.md`).
53
54     ## 4. Design / architecture
55
56     **Reuse map (existing ? `[verified]`, no change for MVP):** `POST /api/query` (list) ?
`POST /api/compile` (stitch) ? `GET /api/highlights/{video_id}/{filename}` (download); `GET
/api/config` (floor) ; `GET /api/jobs/{id}` (carry `result.video_id`). **All net-new code is the
SPA** in `khelsutra` `web/` (Vite + React + TS + Tailwind, per `web/README.md`), plus a small
set of *optional* real-gap engine enablers (§4.3) ? none required for MVP.
57
58     **4.1 Components (net-new SPA)**
59     - **QueryBar + `parseQuery()`** ? a *pure, deterministic, unit-tested* string ?
`{filter, sort}` compiler over `duration` / `ordinal` / `shot_count`. Renders a **parsed-pill**
("filter: duration > 10s · sort: duration desc") so the interpretation is transparent and hand-
correctable; greys out shot-type/player/score tokens with an inline "not available yet". No LLM,
no network call.
60     - **RallyList/Grid** ? one card per `play_segment`: ordinal (by `start_time`), `mm:ss`
start, duration (`end?start`), and a shot-count + confidence chip **only when
`metadata.shot_count` is present** (absent on detector paths that don't write it). Checkbox per
card; the thumbnail area is a placeholder (no endpoint yet).

```

Let me verify the key engine and annotator facts in parallel.

6. user (2026-06-20T10:12:53.389Z)

```
55         parts = key.split(".")
56         obj = config
57         for p in parts[:-1]:
58             obj = getattr(obj, p)
59         setattr(obj, parts[-1], _coerce(val))
60
61
62     def _write_intervals_csv(segments: List[dict], path: str) -> None:
63         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
64         artifact (same schema as the rally-annotator labels)."""
65         with open(path, "w", newline="") as f:
66             w = csv.writer(f)
67             w.writerow(["start", "end", "shot_count", "ending_reason"])
68             for s in segments:
69                 w.writerow([
70                     f'{float(s.get("start_time", 0.0)):.3f}',
71                     f'{float(s.get("end_time", 0.0)):.3f}',
72                     s.get("shot_count", ""),
73                     s.get("ending_reason", s.get("shot_count_confidence", "")),
74                 ])
75
76
77     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
78         with open(path, "w") as f:
79             json.dump({
80                 "video_id": video_id,
81                 "status": status,
82                 "segment_count": len(segments),
83                 "segments": [{"start": float(s.get("start_time", 0.0)),
84                             "end": float(s.get("end_time", 0.0))} for s in segments],
```

7. user (2026-06-20T10:12:54.821Z)

```
120         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
121         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
122
123
124     def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
125                                   frame_width: float = 1280.0, tail_pad: float = 2.0,
126                                   step_px: float = 12.0) -> str:
127         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
128         in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
129         cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
130
131         This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
132         no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
133         learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
134         """
135         rallies: List[Tuple[float, float]] = []
136         with open(labels_csv, newline="", encoding="utf-8") as f:
137             for row in _csv.DictReader(f):
138                 try:
139                     rallies.append((float(row["start_time"]), float(row["end_time"])))
```

```

140         except (KeyError, ValueError):
141             continue
142         duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
143         nframes = max(1, int(duration * fps))
144         base_x = frame_width / 2.0
145         y = frame_width / 4.0
146         os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
147         with open(out_csv, "w", newline="", encoding="utf-8") as f:
148             w = _csv.writer(f)
149             w.writerow(["Frame", "Visibility", "X", "Y"])

```

8. user (2026-06-20T10:12:55.168Z)

```

1      # Output CSV format
2
3      One row per rally. Header is written once when the file is created.
4
5      ```
6      rally_number,start_time,end_time,ending_reason,sport,shots_count
7      1,8.800,11.500,winner,badminton,9
8      2,24.389,46.589,unforced_error,badminton,21
9      3,49.183,54.683,let,badminton,
10     ```
11
12     | Column | Type | Notes |
13     |---|---|---|
14     | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15     | `start_time` | float | rally start, **decimal seconds** from the video start |
16     | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17     | `ending_reason` | enum | `unknown` (default) / `winner` / `forced_error` /
18     `unforced_error` / `service_fault` / `let` / `other` ? see the decision guide in
19     [ENDING_REASONS.md](ENDING_REASONS.md) |
20     | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
21     | `shots_count` | int | **optional** ? number of shots/strokes in the rally. **Blank**
22     when not entered (as in row 3 above); added in v1.6. Older CSVs without this column still load.
23
24     Notes:
25     - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
26     - Rows are appended live; **Undo last** truncates the last row (one level).
27     - Consumers should read by **column name** (the header is stable but column *order* may
28     evolve; extra
29     columns may be added later). Most CSV `DictReader`-style loaders already tolerate
30     extra columns.
31
32     ## Ingesting the labels
33
34     This CSV is intentionally compatible with common rally-segmentation tooling that reads
35     `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
36
37     ```bash
38     # Generic: any tool that takes a labels CSV of start/end seconds.
39
40     # Example ? score a detector against these labels, or slice clips by them
41     # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
42     python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
43     --fps 59.94 --frame-width 1920
44     python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
45     --rallies all
46
47     # Example ? register as curated golden data in sports-data-collector:
48     python -m src.cli.curate import-local --sport badminton --video-path match.mp4
49     --annotations-path match.rallies.csv
50     ```
51
52

```


44 The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
45

9. assistant (2026-06-20T10:12:57.548Z)

The research is well-grounded and matches the actual files. Let me verify the gt_loader and the site structure quickly.

10. user (2026-06-20T10:12:58.989Z)

```
14 Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15 - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16   columns ? rally_number, ending_reason, sport, ? ? are ignored);
17 - no header: columns 0,1 positionally.
18 Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
(``annotations.parse_timestamp``).
19 ""
20 import csv
21 import logging
22 from typing import List, Optional, Tuple
23
24 from backend.eval.annotations import parse_timestamp
25
26 logger = logging.getLogger(__name__)
27
28 Interval = Tuple[float, float]
29
30 _START_NAMES = ("start", "start_time")
31 _END_NAMES = ("end", "end_time")
32
33
34 def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35     """If `row` is a header naming start/end columns, return their indices, else
None."""
36     cells = [(c or "").strip().lower() for c in row]
37     start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38     end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39     if start_idx is not None and end_idx is not None:
40         return start_idx, end_idx
41     return None
42
43
44 def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45     """Load GT rally intervals from either golden-CSV convention.
46
47     strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48     (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50     skipped ? a silently shrinking GT set flatters precision.
51
52     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53     (they break one-to-one matching and usually indicate a labeling error).
```

11. user (2026-06-20T10:13:00.375Z)

```
42 async function send(res, response) {
43     res.statusCode = response.status;
44     for (const [k, v] of response.headers) res.setHeader(k, v);
45     res.end(Buffer.from(await response.arrayBuffer()));
46 }
47
48 async function serveStatic(pathname, res) {
49     const rel = pathname === "/" ? "index.html"
50       : pathname === "/capture" ? "capture.html"
```

```

51         : pathname.replace(/^\/+/, "");
52     const file = normalize(join(PUBLIC, rel));
53     if (!file.startsWith(PUBLIC)) { res.statusCode = 403; return res.end("forbidden"); }
54     try {
55         const data = await readFile(file);
56         res.setHeader("content-type", TYPES[extname(file)] || "application/octet-stream");

```

12. user (2026-06-20T10:13:00.432Z)

```

1     # khelsutra docs index
2
3     ## Tracked work (this repo)
4     - **`PER_RALLY_SELECTION.md`** ? interactive pick-rallies ? reel builder (RallyPicker).
The alpha
5     SPA over the engine's rally index; hybrid grid + deterministic query bar on the
**existing**
6     `/api/query` + `/api/compile` contract. `DRAFT` (owner-gated on scope).
7     - **`REQUEST_ACCESS_FORM.md`** ? segmented lead-capture form + Worker/D1 backend
(shipped).
8     - **`PORTABILITY.md`** ? the Node+SQLite VM-host runbook (Cloudflare ? droplet parity).
9
10    ## Engine planning docs
11    Canonical planning docs currently live in the engine repo (`badminton-highlight-
indexer/docs`):
12    `UI_PROPOSAL.md` · `HOSTING_PLAN.md` · `UI_ALPHA_EXECUTION_PLAN.md`. They migrate here
when the
13    control plane lands (per ADR-009 in the engine repo's
`docs/archives/decisions/DECISIONS.md`).
14

```

13. assistant (2026-06-20T10:13:04.216Z)

All facts verified against the actual code. The research is accurate. I have everything needed to produce the consolidated design doc. Let me write the structured output.

14. user (2026-06-20T10:16:50.568Z)

Structured output provided successfully

15. assistant (2026-06-20T10:17:00.265Z)

Done. I verified the load-bearing facts against the actual code before consolidating ? specifically the schema bridge (`stub\_runner.py:139` reads `row["start\_time"]`/`row["end\_time"]` via DictReader; `gt\_loader.py:30-31` accepts both `start`/`start\_time` aliases; the annotator header at `CSV\_FORMAT.md:6` matches exactly), the train/serve wall and ship-gate (`\_\_main\_\_.py`, `harness.py`), and the khelsutra site structure (`server.js:48-51` clean-URL pattern, `web/` has no router).

The consolidated design doc and all structured fields are returned via StructuredOutput. Key honest conclusions:

- **Placement:** ONE marketing page (`site/public/own-model.html`, slug `own-model`) with three anchored sections ships now; the SPA correction/contribute screen is design-tracked + owner-gated (depends on the unbuilt `PATCH /api/segments/{id}`). No "Train now" button anywhere.
- **Schema bridge:** annotator CSV matches the engine label path with **no transform**; the only real prerequisite is the MD5 + filename-stem join (do not re-encode the video after annotating).
- **Brutal honesty preserved:** annotate half is real/shipping (MIT VLC plugin, v1.6.4); train half is an open CLI whose GPU-free path is a plumbing mock and whose real model needs a GPU + in-flight native WASB + an owner-gated ship-gate; paid LLMs are never a training source; no accuracy claim vs paid LLMs is supportable.
- **Screenshots:** all five are TO-CAPTURE (the repo has zero committed images) ? never fabricate.